

Diffusion Models from Scratch

Building a Class-Conditional Image Generator

Presenter: Arhan Khade

Course: Seasons of Code 2026

Project: Class-Conditional EMNIST Letters Generator

Project Overview & Motivation

The Goal

- Demystify generative AI by implementing every component from first principles.
- Move from standard MNIST digits to a larger, more complex dataset of handwritten characters (EMNIST Letters).

Core Components

- Custom U-Net with skip connections.
- Multi-schedule noise process.
- DDPM & DDIM sampling loops.
- Classifier-Free Guidance (CFG).

Learnings: Building the ML Engine

- **Autograd & Graph:** Deconstructed what happens during `loss.backward()` and how gradients flow through layers.
- **Custom Training Loops:** Bypassed high-level PyTorch Lightning/HF wrappers to write raw PyTorch optimization loops.
- **Hardware Acceleration:** Managed device transfers (`.to(device)`) between CPU, MPS (Apple Silicon), and CUDA.
- **Optimization Dynamics:** Observed convergence differences between SGD and Adam optimizers on pixel-space losses.

Learnings: U-Net Convolutional Backbone

Spatial Dynamics

- **Downsampling:** Encoder blocks compress resolution to extract high-level context.
- **Upsampling:** Decoder blocks reconstruct pixel arrays.
- **Skip Connections:** Concatenate encoder maps to decoder maps.

The Skip Connection Advantage

- Prevents vanishing gradients across deep convolutional paths.
- Preserves fine-grained pixel coordinates of character strokes.
- **Conditioning:** sinusoidal time embeddings injected before activation.



Learnings: The Forward Diffusion Process

- **Closed-Form Sampling:** Sampling x_t directly from x_0 at any timestep t using cumulative product coefficients $\bar{\alpha}_t$:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon \quad \text{where } \epsilon \sim \mathcal{N}(0, I)$$

- **Linear Schedule:** Constant beta values. Fast noise injection, but can destroy early details too aggressively.
- **Cosine Schedule:** Maintains stable signal-to-noise ratio longer, preserving shape structure at lower timesteps.

Learnings: The Reverse Process (DDPM)

Denoising Objective

- The U-Net is trained to **predict the added noise** ϵ_t , not the denoised image x_0 .
- Time embeddings condition the network at every layer, informing it of the current noise scale.

Iterative Generation

- Progressive denoising over $T = 1000$ steps.
- Each step subtracts a fraction of the predicted noise and injects standard Gaussian noise to maintain correct variance.

⚡ Implementation: Fast Sampling with DDIM

DDPM (Stochastic)

- **Sequential Markov Chain:** Must step through all 1000 steps during inference.
- High quality, but slow generation speed.

DDIM (Deterministic)

- **Non-Markovian Process:** Bypasses sequential step restrictions.
- Allows skipping timesteps (sampling on a sub-sequence).
- **Speedup:** Reduced steps from 1000 to 25–50 with no loss in visual quality.

Implementation: Classifier-Free Guidance

- **Class Conditioning:** Added class label embeddings (`nn.Embedding(num_classes + 1, emb_dim)`) to U-Net layers.
- **Joint Training:** Dropped class conditioning with 15% probability to train both unconditional and conditional paths.
- **Guidance Interpolation:** Extrapolating the noise prediction during sampling:

$$\epsilon_{\text{cfg}} = (1 + w)\epsilon_{\text{cond}} - w\epsilon_{\text{uncond}}$$

- **Scale (w):** Balancing letter sharpness (fidelity) and writing style variety (diversity).



Implementation: EMNIST Letters Pipeline

- **Dataset Scale:** EMNIST Letters split ($\sim 124,800$ images, 28×28 grayscale, 26 classes A–Z).
- **Transposition Correction:** Standard EMNIST is stored rotated and mirrored. Corrected in `dataset.py` using:

```
transforms.Lambda(lambda img: img.transpose(PIL.Image.TRANSPOSE))
```
- **Label Alignment:** Shifted 1-indexed EMNIST labels (1–26) to 0-indexed classes (0–25).
- **Geometric Augmentation:** Random rotations ($\pm 10^\circ$) and translations (10%). Flips were disabled to avoid letter corruption (e.g. 'b' to 'd').

⚙️ Implementation: Training Run Specifications

Hyperparameters

- **Epochs:** 25
- **Batch Size:** 128
- **Learning Rate:** 1e-4 (Adam)
- **Base Channels:** 64
- **U-Net Depth:** 3
- **Timestep Embeddings:** 256

Run Details

- Trained on a remote GPU machine (conda `llmboi` environment).
- **Robust Logging:** Logs saved locally to `results/experiment_log.csv` and synced programmatically to Weights & Biases.



Results: Loss Curves and Sample Generation

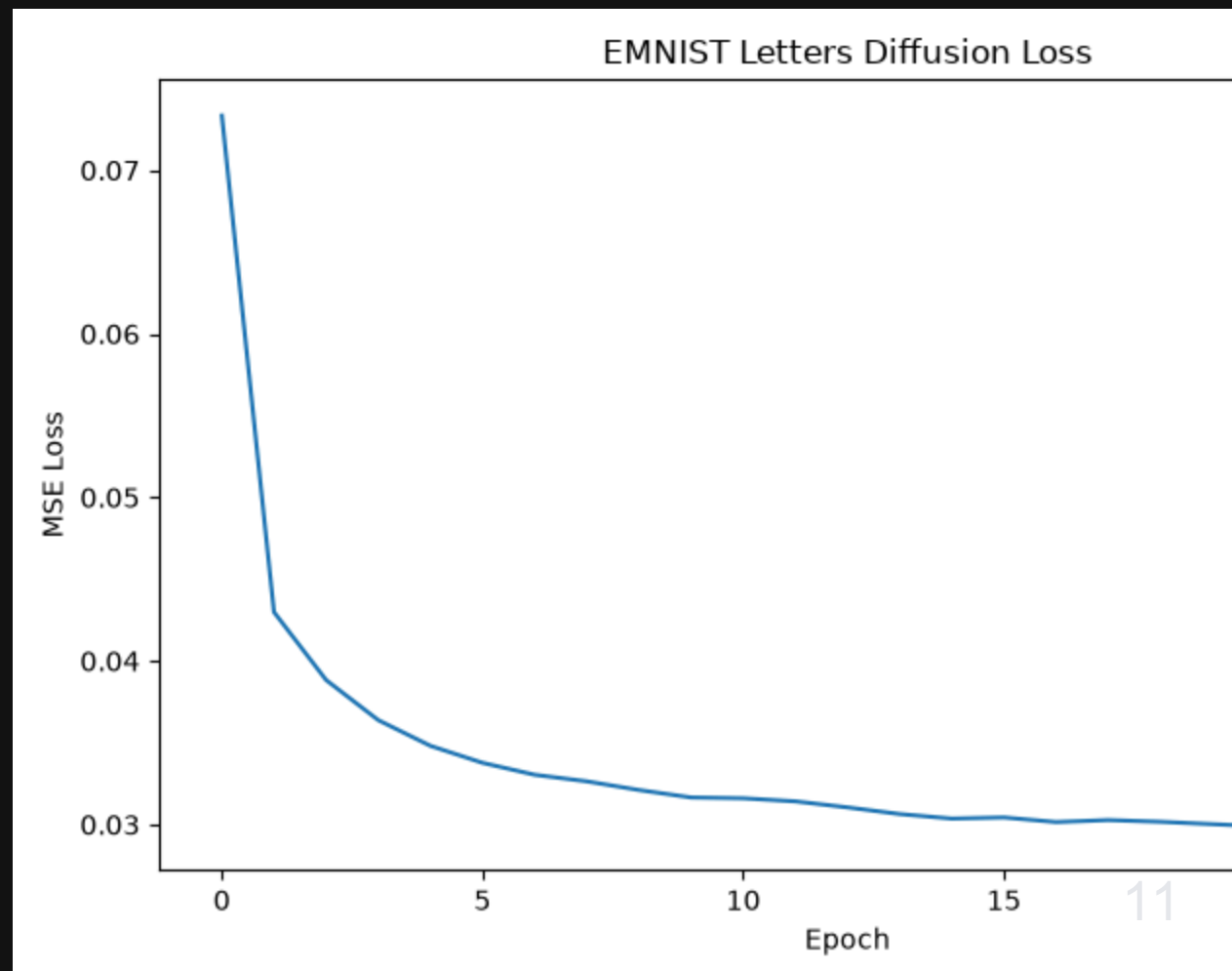
Loss History & Convergence

- Steady, smooth MSE loss decay indicating consistent training.
- Stabilized around **0.034** after epoch 5.
- Training log: `experiment_log.csv`

Sample Generation

- Legible shapes for characters

Visual Progress



Results: Guidance Scale Analysis

- $w = 0.0$ (**Unconditional**): Generates blurry, unstructured letter-like shapes without adhering to a specific target class.
- $w = 1.0$ (**Weak Guidance**): Identifiable letters, but stroke lines are soft and have high geometric variance.
- $w = 3.0$ (**Sweet Spot**): Sharp, highly legible character margins and consistent pen strokes.
- $w \geq 5.0$ (**Over-guidance**): Saturates pixel values, creating thick, high-contrast, bold letters.



Future: Distributed Compute & Deeper Models

1. Scaling Compute

- **PyTorch DDP:** Implement Distributed Data Parallel to train across multi-GPU setups.
- **EMA Logging:** Add Exponential Moving Average of model weights to smooth generated letter margins.

2. Modern Architectures

- **Bottleneck Attention:** Add self-attention blocks in the bottleneck to capture global stroke coordination.
- **Diffusion Transformers (DiT):** Replace convolutional U-Net with a ViT-based backbone.

Future: Latent Space & Text Prompting

- **Latent Diffusion (LDM):** Train an Autoencoder (VAE) to compress images into a low-dimensional latent space. Conduct diffusion in the latent space to dramatically reduce compute requirements (like Stable Diffusion).
- **Text-to-Image (CLIP):** Replace class categorical embeddings with contextual text embeddings from a pre-trained CLIP text encoder to support full descriptive prompts.

Reflections & Course Feedback

- **Hands-on Philosophy:** Writing training loops, UNet blocks, and schedules from scratch is vastly superior to importing libraries—it builds deep intuition.
- **Milestone Structure:** Weekly check-ins and clean assignments provided steady, manageable progression.
- **Recommendations:** Keep the "from first principles" approach; it demystifies complex ML research papers.

Thank you! Questions?